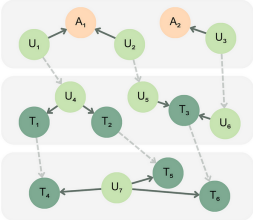
Prof. Rachlin needs new music!

DS 4300: Large-Scale Storage and Retrieval (Prof. Rachlin)



Problem Description:

A band I really like is *The Strokes,* particularly their album: *Is This It.* In fact, I'm listening to this album as I type this! I'm also a huge fan of Regina Spektor. I think she is a musical genius. Her song *Us* has been my favorite song for years! But my 19-year-old daughter, Abby, thinks it is high time for me to expand my musical tastes, and I'd like you to help me by using a graph database (Neo4J) to build a general-purpose song recommendation system.

I downloaded Spotify data from Kaggle containing information on about 114,000 songs and their musical properties. The dataset includes fifty-two songs by the Strokes including all the songs on the *Is This It* album. There are also several songs by Regina Spektor- I like them all! Model some or all the Spotify data as a graph database. Nodes are songs and their properties. Edges connect *similar* songs, where similarity is defined <u>however you like</u>. Then, using a cypher query, or a series of cypher queries, generate FIVE song recommendations for Professor Rachlin. Do NOT recommend any songs by *The Strokes* or *Regina Spektor.*

Some Hints and Guidelines.

This is a very large dataset, so connecting every pair of songs via an edge with a similarity score property is probably not feasible. Your graph database would have $(114{,}000)^2$ = 13 billion edges. Instead, you are free to sample the dataset any way you like but be sure that your sample includes some or all of the songs by The Strokes and Regina Spektor. Connect any two songs via an edge if they meet some similarity metric threshold. For example, you might sample 1000 songs, and pre-compute a Euclidean distance metric between every pair of songs, including an edge between two songs if those two songs are sufficiently similar.

IMPORTANT CLARIFICATION: I am asking you to build a <u>general-purpose song recommendation</u> system that could recommend songs based on liking other songs in the database. You will then *test* your approach by recommending songs for me based on the fact that I like The Strokes and Regina Spektor. Do not simply build a song recommendation capable of *only* recommending songs for me. For example, building a limited network of similar songs connected only to the songs in the *Is This It* album would not be very useful to Spotify or other streaming music

services! By building a larger network it also opens the possibility of exploring the differences in genre-specific sub-networks. Are the patterns of interconnection between Hip-Hop songs different from those of Pop? If we built such a network to include release dates, could the resulting graph give us insights into the history of popular music or musical trends and influences?

What to Submit

1) **Your documented code.** Including any pre-processing or sampling code (implemented in a programming language of your choice) and all cypher queries that you executed to build your graph network and generate the recommendations.

   Code documentation should be sufficient to reproduce your results, including:

   a) How you sampled the data

   b) The full graph data model: Specifically, what are your node and relationship properties and labels?

   c) Explanation of your recommendation algorithm

2) **Poster Slide: ONE slide (in PDF format, not PPTX!)**.

   a) A graph visualization connecting songs by the Strokes to songs to your recommendations. In other words, a graph representation showing the main results of your queries.

   b) A 1-paragraph summary description of your recommendation approach.

   c) Identifies the TOTAL SIZE of your song graph model: # nodes, # edges

   d) A list of the FIVE song recommendations (Artist, Album, and Title).
   Post your recommendations on the shared Google Sheet (provided on the Canvas Assignment.)

   e) Your song similarity metric. (What is your rule for connecting two songs?)

   f) We'll share our posters and listen to (what I hope will be) a lot of good music in an upcoming class.